AI-Assisted Heterogeneous Computing for Dynamic Task Scheduling and Offloading: A Review of Publications, Patents, Products and Technology Trends

Lakshay S, Arkkadu Bharath Reddy
Reg. Nos. 26MAI0064, 26MAI0058 — Course: Computer Architecture and Organization
Guide: Dr. Yokesh Babu S, Associate Professor Grade 1. [Institution, department and e-mail to be filled by the authors]

***Abstract*—Modern processors, GPUs and edge devices differ in speed, power and availability, so deciding *where* and *in what order* the tasks of a dependency graph run has become a first-order performance problem. This review surveys the field for the project "AI-assisted heterogeneous computing framework for dynamic task offloading and scheduling". We (i) define the domain and substantiate the problem with verified statistics on data-centre energy, accelerator adoption, edge growth and measured under-utilisation; (ii) compare 13 recent publications on DAG scheduling and dependent-task offloading [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13] along method, workload, platform, metric and limitation; (iii) analyse the base patent US 11,561,826 B1 at claim level and six related patents from IBM, Intel, Samsung, Apple and Google; (iv) compare ten company and open-source products that schedule work on heterogeneous resources; and (v) summarise technology trends. Three findings structure the remaining work: nearly all learned schedulers are evaluated in simulation, almost none reports its own decision overhead or a statistical interval, and none compares against an *informed* simple rule. These gaps motivate a measured, overhead-accounted study on real hardware; the closing section records how the project's final benchmark of static plans against dynamic dispatch on a CPU+GPU laptop answers each gap.**

***Index Terms*—*heterogeneous computing, DAG scheduling, task offloading, edge computing, reinforcement learning, runtime prediction, patents, survey*

I. Introduction: Domain and Scope

Heterogeneous computing combines processing elements with different speeds, instruction sets, memories and power behaviour in one system: performance (P) and efficiency (E) cores in a laptop processor, a discrete GPU, an FPGA, a single-board computer such as a Raspberry Pi 5 [14] (four Arm Cortex-A76 cores at 2.4 GHz), or a remote edge or cloud node. Applications are rarely a single kernel. They are *workflows* or *task graphs*: directed acyclic graphs (DAGs) whose nodes are tasks and whose edges are data dependencies, for example the Montage astronomy mosaic workflow [15], [16], image-analysis pipelines, or the job stages of a data-processing cluster [17].

A *scheduler* decides, whenever tasks become ready, (a) the **order** in which ready tasks are served and (b) the **placement** of each task on a worker. *Offloading* is the special case in which the candidate worker is on another device and a transfer cost must be paid. Both decisions depend on three kinds of information: the structure of the graph, the state of the resources (busy, idle, loaded, reachable), and an estimate of how long each task takes on each resource. Classical list schedulers such as HEFT and PEFT [18], [19] assume the estimates are given; learned schedulers [1], [7], [10] try to infer a policy from interaction; runtime systems such as StarPU [20] calibrate per-task performance models.

This review is the first deliverable of the project *AI-assisted heterogeneous computing framework for dynamic task offloading and scheduling*. Its research focus is how much prediction of task execution time is needed to improve real scheduling, once the cost of prediction and of the scheduler itself is counted. Section II substantiates the problem with statistics. Sections III to V compare publications, patents and company products. Section VI summarises technology trends and their adoption status, and Section VII lists the gaps and records the status of each after the final benchmark. The final benchmark asks a sharper version of the question: does a *dynamic* scheduler, which decides while the workflow runs, finish sooner than a *static* plan computed beforehand from the same cost model, on a hybrid CPU+GPU workstation?

II. Problem Substantiation with Statistics

Four independent trends make scheduling on heterogeneous resources important. Each figure below was checked against its source; where only a secondary report was available it is marked.

TABLE I
VERIFIED STATISTICS THAT MOTIVATE THE PROBLEM

Indicator	Value	Source
Data-centre electricity use	about 415 TWh in 2024 (about 1.5% of global electricity); projected about 945 TWh by 2030 in the IEA base case; AI-optimised facilities more than quadruple	[21] (secondary reports of the IEA report)
Accelerated HPC systems (TOP500, Jun. 2025)	232 of 500 systems (46.4%) use accelerators; they hold 59.7% of cores and about 86.2% of peak FP64 flops	[22] (secondary analysis)
Connected IoT devices	18.5 billion at end-2024 (+12%); 21.1 billion in 2025 (+14%); about 39 billion forecast for 2030	[23]
Under-utilised clusters	Alibaba co-located cluster (1,313 machines): over 80% of the time at 10–30% CPU use; Google production traces: 20–40% most of the time	[24] (read in full text)
Hybrid cores in client CPUs	Intel Thread Director listed for 12th, 13th and 14th generation Core processors (11 product families)	[25]

The first three rows show that the *amount* of heterogeneous capacity is large and growing: electricity demand rises with AI workloads, accelerated systems dominate the peak performance of the largest machines, and tens of billions of edge devices produce data that may have to be processed close to where it is created. The fourth row shows that capacity is often *wasted*: a cluster that spends most of its time at 10–30% CPU use is not limited by hardware but by how work is placed. Scheduling quality therefore translates directly into energy, cost and latency.

Measured heterogeneity on our test machine. The project test bed is a laptop with an Intel Core i5-13500HX (6 P cores, 8 E cores), an RTX 3050 6 GB GPU and 15.7 GB RAM running native Windows. Fig. 1 (left) shows the median kernel time of three image-pipeline stages pinned to one P and one E core: a 512-pixel denoise task with 30 iterations takes 156 ms on a P core and 560 ms on an E core, i.e. 3.6 times longer; the two shorter stages are 2.8–3.3 times slower. On the GPU (right) the same convolution-type stage is up to 15 times faster at 1024 pixels but only breaks even at small tiles, because host-device copies dominate. A scheduler that ignores these ratios sends heavy work to slow cores.

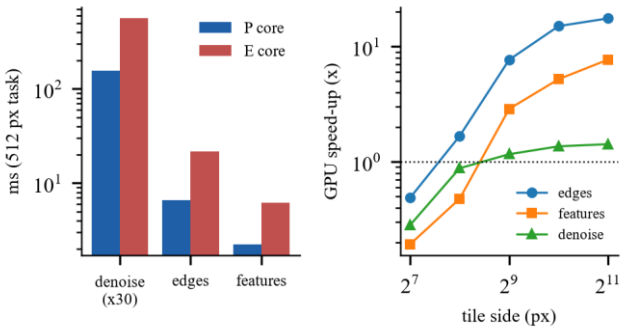


Fig. 1. Measured heterogeneity on the project machine. Left: median kernel time of one stage task on a P and an E core (log scale). Right: GPU speed-up of three stages versus tile size (CPU/GPU outputs equal within tolerance). Source: project profile and GPU spike files.

Measured cost of blind placement. On a workload of 64 independent image chains in which 4 to 40 chains are 60 times more expensive in one parameter, a blind greedy dispatcher (first idle eligible worker) needed 5% to 37% more time than a rule that merely knew which chains were heavy (25% to 37% when 4 to 16 chains were heavy) (paired, 12–36 blocks per setting, 95% intervals in the project report). On the 59-task Montage mosaic the first nine reprojection tasks take about 98% of the 1.46×10^3 s sequential run, so any placement error in those nine tasks dominates the result. Section III shows that the published learned schedulers do not measure situations of this kind on real hardware.

III. Survey: Comparative Analysis of Existing Solutions

A. Method of the survey

We analysed the 12 papers supplied as base papers (full text read) and one further paper at abstract level [8], plus classical and related work used for positioning. For each paper we extracted the algorithm, workload, platform, metrics, reported result and limitations, and verified claims against the PDF. A keyword check over the 12 full texts found no occurrence of "confidence interval", HEFT appears in five papers and PEFT in two, and measured decision (inference) time is mentioned in only two.

TABLE II

Comparison of recent solutions (DAG scheduling and dependent-task offloading)

Ref.	Method	Workload / data	Platform	Reported result	Limitation for our goal
[1]	GNN + RL selects task; DEFT with task duplication places it	TPC-H DAG jobs	Simulated cluster, 50 executors	up to 26.7% makespan, 35.2% speedup vs. 7 baselines	simulation; speed-only heterogeneity; no variance test
[2]	Dueling double DQN, MDP	Google Cloud Jobs, Alibaba trace	Trace-driven simulation	shorter response time, better load balance	cluster level; DRL cost; no hardware
[3]	Off-policy RL + seq2seq	Synthetic DAGs	MEC simulation	approaches optimal latency/energy	synthetic; simulated edge
[4]	FastFit / Multi-FastFit lightweight scheduling	4 benchmarks	Zynq UltraScale+ CPU+FPGA (real)	up to 2x over an auto-tuned scheduler; overhead measured	FPGA specific; no DAG, no prediction
[5]	Bidirectional GCN + FC net; OCT + duplication	heterogeneous DAG scenarios	Simulation	better than compared DAG schedulers	heavy model; simulation
[6]	DTOS: parallel DNNs, priority replay	Synthetic DAGs, dynamic channel	MEC simulation	lower delay and energy	network simulation only
[7]	SATA-DRL (DQN), MDP	25-node Montage task graphs (simulated)	Simulated MEC	lower makespan and deadline misses	simulation; no real Montage run
[8] (abstract)	GAT + DRL, multidiscrete actions	Synthetic distributed data	Simulated MEC	adapts to simple and complex MEC	synthetic; heavy model
[9]	Distributed actor-critic, binary offloading	2 simulated scenarios	5G edge-cloud (simulated)	better QoE / energy	no DAG dependencies
[10]	ADPRL: dependency-aware reward, joint bandwidth	Alibaba trace + synthetic	Simulation + testbed with 1 Jetson Xavier NX and 2 Jetson TX2	lower completion time and energy	network-flow focus; DRL cost
[11]	Meta-RL + seq2seq + NSGA-II	Simulated DAG tasks	Simulated edge	21% less delay, 19% less energy	simulation; complex model
[12]	DAG-DQN	Vehicular applications	Simulated IoV	16% less time, 19% less energy, 70% faster decisions vs. heuristic	IoV simulation only

Ref.	Method	Workload / data	Platform	Reported result	Limitation for our goal
[13]	GCN-Transformer + PPO; ECT replication heuristic	DAG applications	Simulated multicore	11.09% lower time and power cost	simulation; heavy model

B. Taxonomy and comparison

The papers fall into four families. (1) **Learned selection plus a placement heuristic** [1], [5], [13]: a GNN or Transformer picks the next task, a classical rule (OCT, earliest completion time, task duplication) picks the processor; this shrinks the action space but keeps the heuristic as a crutch. (2) **End-to-end DRL for offloading** [2], [3], [6], [7], [8], [9], [10], [11], [12]: MDP formulations with DQN, actor-critic or PPO; the focus is latency and energy under dynamic network conditions. (3) **Lightweight runtime scheduling** [4]: low-overhead asynchronous chunk scheduling on a real CPU+FPGA board, without learning. (4) **Prediction-driven list scheduling** [26], [20], [18], [19]: per-task runtime models feed HEFT-type or StarPU schedulers. Table III compares the survey on the properties that matter for a real prototype.

TABLE III

PROPERTY MATRIX OF THE SURVEYED SOLUTIONS (✓ = PRESENT IN THE PAPER, – = NOT FOUND)

Property	1	2	3	4	5	6	7	9	10	11	12	13
DAG dependencies	✓	–	✓	–	✓	✓	✓	–	✓	✓	✓	✓
Physical hardware evaluated	–	–	–	✓	–	–	–	–	✓	–	–	–
Learned model in decision loop	✓	✓	✓	–	✓	✓	✓	✓	✓	✓	✓	✓
Decision/inference time reported	✓	–	✓	✓	–	–	–	–	–	–	✓	–
Compares HEFT-type list scheduler	✓	✓	✓	–	✓	–	–	–	–	–	–	✓
Reports confidence intervals	–	–	–	–	–	–	–	–	–	–	–	–
Informed simple-rule baseline	–	–	–	–	–	–	–	–	–	–	–	–

The matrix is derived from keyword and reading checks of the supplied PDFs (HEFT/PEFT mentions, “confidence interval”, “inference time”, “testbed”); the [8] column is filled from its abstract only. Two patterns stand out. First, **only two works touch physical hardware** (a CPU+FPGA board in [4]; a three-board Jetson testbed in [10]), and neither schedules a real scientific workflow. Second, **no work compares against an informed non-learning rule**, so the benefit that is attributed to learning cannot be separated from the benefit of simply knowing which tasks are expensive. Reported gains range from 11% to 27% (Fig. 2) but come from different baselines and simulators and are not comparable with one another.

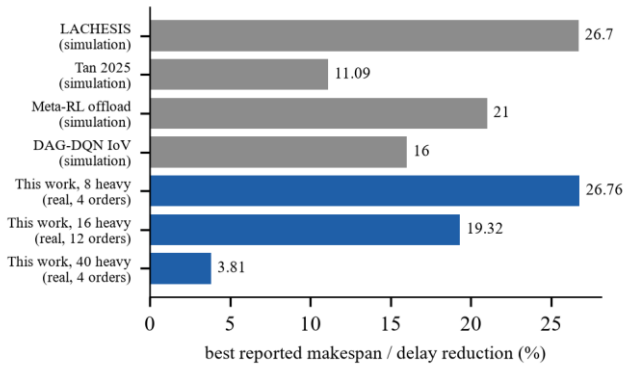


Fig. 2. Best reported reductions in the surveyed papers (grey, all simulation) next to this project’s measured values (blue, real hardware). Baselines differ between bars; the figure shows scale, not a ranking.

C. Related prediction and positioning work

Prediction-driven scheduling is not new. [26] predicts workflow task runtimes with Bayesian regression and reports that HEFT driven by its predictions stays within a few percent of HEFT with perfect knowledge (simulation). [27] study how runtime-estimate accuracy changes workflow scheduling and, according to their abstract, find that under high utilisation the order in which work is processed matters more than estimate accuracy; we consider this a result to test, not to assume. StarPU [20] has used history-based performance models for CPU/GPU DAG scheduling for over a decade. A 2026 preprint [28] places AI tools on GPU or CPU with an LLM agent and a runtime monitor on real hardware and compares against HEFT and StarPU, but on DAGs of about four tools.

D. Static versus dynamic scheduling

The classical list schedulers HEFT and PEFT [18], [19] are *static*: they compute, before execution, a processor and a start time for every task from a cost matrix. A static plan commits to its estimates; when a task finishes earlier or later than planned, the assignment does not change. *Dynamic* schedulers decide when a worker becomes idle or a task becomes ready. StarPU [20] is the established runtime of this kind for CPU/GPU task graphs and calibrates history-based performance models; the learned schedulers of Section III are dynamic as well. Two observations from the survey matter for the project. First, HEFT is used as a baseline in five of the 13 papers, always in simulation, so the cost of *planning itself* and the plan’s error against real execution are not observed. Second, we found no paper that gives the static and the dynamic scheduler the *same* runtime model and measures both end to end, including start-up and scheduling time, which is the comparison needed to say what dynamic decisions add. The final benchmark of the project (Review 3) is designed for this comparison.

E. Strengths and weaknesses by family

Learned selection plus heuristic placement is the most mature family for DAG scheduling: [1] reports that its decision step fits inside tens of milliseconds, and [13] reduce the action space by separating task selection from processor allocation. Its weakness is dependence on the expert rule for placement and on training data from a simulator; neither paper shows the trained policy on a machine whose speed ratios were measured. *End-to-end DRL for offloading* optimises latency and energy under changing channels [6], [10], [11]; it is well suited to wireless settings in which the network is the bottleneck, but the evaluated task graphs are small or synthetic and the cost of training and inference on the device that must decide is rarely quantified. *Lightweight runtime scheduling* [4] is the only family that measures overhead explicitly and shows why it matters: for fine-grained work the scheduler can cost more than it gains. *Prediction-driven list scheduling* shows that moderate prediction error is tolerable in simulation [26] and that order can dominate accuracy [27]; what is missing is a real-execution test of that claim.

F. Evaluation practice

Evaluation practice in the base set is uneven. Baselines are mostly random, round-robin, FIFO or greedy dispatchers and, in five papers, HEFT; none includes a rule that exploits the same information a learned model would learn. Results are reported as single percentages (e.g. 26.7%, 21%, 19%, 16%) obtained on simulators with different cost models, so that the numbers cannot be pooled or ranked. Repetition counts are seldom stated and no paper reports interval estimates. For a project whose aim is to say *how much prediction is enough*, these are the practices to improve: real execution, interleaved repetitions, paired intervals, overhead accounting and informed baselines.

IV. Survey: Comparative Analysis of IPRs (Patents)

The base patent of the project is US 11,561,826 B1, *Scheduling processing of machine learning tasks on heterogeneous compute circuits*, assigned to Xilinx, Inc. (inventors S. Nagpal, A. Karumannil, V. Jain, A. K. Patil; application 17/096,136 filed 12 Nov. 2020; granted 24 Jan. 2023; 20 claims, 8 drawing sheets; CPC G06F 9/4881, G06F 16/9024, G06N 3/08) [29]. We read the granted document (images of the claims pages) and searched for related grants on Google Patents. Legal status and expiry dates quoted below are Google Patents assumptions, not legal conclusions, and no freedom-to-operate opinion is implied.

A. Claim-level analysis of the base patent

Independent claim 1 (method) and claim 14 (system) recite: instantiating *kernel objects* of different *kernel types*, each type indicating a compute circuit of a heterogeneous plurality; generating a graph whose nodes are tasks assigned to one or more kernel objects and whose edges are data dependencies; creating task queues and assigning kernel objects to them; *enqueueing* tasks by threads executing the kernel objects; and *dequeueing* tasks by those threads and activating the compute circuits to process them. Dependent claims add blocking versus non-blocking kernels (claims 2, 15), several graphs feeding one queue (3, 16), two kernel objects of the same or different types sharing queues (4, 5, 17, 18), multi-threaded kernels with a configurable thread count (6–7), **measuring circuit performance and adjusting the number of threads** (8), output-buffer addresses and **in-degree counters decremented on parent completion** (9–10), utilisation-triggered thread increase (11), a report function for kernel statistics (12) and concurrently generated graphs (13). The claims do not recite machine learning, prediction of execution time, a transfer-cost model or offloading to a remote device; the title refers to machine-learning *workloads* being scheduled, not to a learned scheduler. Table IV places the project against this and six other grants.

TABLE IV

Comparative analysis of patents (IPRs)

Patent	Assignee, dates	What it claims or discloses	Prediction?	Relation to our work
[29] US 11,561,826 B1	Xilinx; filed 2020-11-12; granted 2023-01-24	Task graph; kernel objects tied to compute circuits; per-kernel task queues; threads enqueue/dequeue; in-degree gating; performance-based thread adjustment	No	Base work; mapped to eligibility-gated greedy dispatch (baseline B4)
[30] US 11,740,933 B2	IBM; filed 2020-07-01; granted 2023-08-29	Meta pre-processor receives a DAG; a *learning agent assigns a rank* to a task; scheduler runs tasks in rank order on a heterogeneous SoC	Learned rank	Closest: rank-ordered dispatch; no idle-worker-only placement or log-derived parameter model in the claim summary
[31] US 11,966,776 B2	IBM; filed 2021-07-14; granted 2024-04-23	ML mechanism learns DAG policies, decides whether to drop DAGs, schedules tasks using priorities and *task execution estimates*	Yes (estimates)	Learned DAG policy with drop decisions; deadline-oriented
[32] US 10,186,007 B2	Intel; filed 2014-12-26; granted 2019-01-22	CPU+GPU: profiling threads time a portion of work on each device, compute a distribution ratio, split the remaining work	Online profiling	Per-run profiling, no DAG, no learned model
[33] US 10,162,671 B2	Samsung; filed 2013-04-11; granted 2018-12-25 (lapsed 2022)	Predict each task's duration when the state of a core changes, using solo-vs-concurrent duration ratios; schedule on heterogeneous cores	Yes	Contention-aware duration prediction; terminal devices
[34] US 11,775,811 B2	Apple; filed 2019-01-08; granted 2023-10-03	Order algorithms by temporal dependencies, build a graph whose nodes name a processor (CPU, GPU, neural processor), execute by traversing for concurrency	No	Static graph-to-processor mapping
[35] US 11,544,113 B2	Google; filed 2019-12-19; granted 2023-01-03	Determine resources of an ML workload, number of hosts per accelerator, task specification per host NUMA topology	No	Cluster ML placement, not DAG runtime

Observations. (i) Task graphs, eligibility-gated queues and utilisation-based scaling are claimed by the base patent and

cannot be presented as new. (ii) *Rank-ordered dispatch with a learned agent* is disclosed by IBM's 2023 grant; any claim to "learning a task priority" must be examined against it. (iii) *Duration prediction for heterogeneous cores* appears in Samsung's (lapsed) patent and, as task execution estimates, in IBM's second grant. (iv) None of the seven documents, as summarised, describes an empirical method for deciding *when prediction is worth its cost* or a decision-overhead accounting. The project therefore treats B4 as a comparison baseline, not as its own invention, and positions its contribution as an evaluation methodology and a measured study; any patent filing would need a professional prior-art search first.

V. Survey: Comparative Analysis of Company Products

Ten products and platforms that decide where work runs were compared on documentation pages (Table V). We record only what the cited page states; where a page did not describe a feature we write "not stated".

TABLE V
COMPANY AND OPEN-SOURCE PRODUCTS THAT SCHEDULE ON HETEROGENEOUS RESOURCES

Product (vendor)	Scope	Heterogeneity handled	Dependencies	Prediction / learning	Note
Intel Thread Director [25]	Hybrid client CPU	P vs E cores; hardware tracks instruction mix per thread at nanosecond granularity and feeds the OS	No (threads)	Hardware classification; Intel states it uses machine learning	12th–14th gen Core; no benchmark figures on the page
Windows QoS [36]	OS scheduler	QoS levels may restrict threads to a subset of processors or prefer a class (e.g. EcoQoS to efficient cores)	No	Heuristics + developer hints	Priority remains the main metric
Linux EAS [37]	Kernel	Asymmetric CPU capacity (big.LITTLE); energy model	No	None; energy-model lookup	Disabled above 80% utilisation; no symmetric topologies
AWS Batch [38]	Cloud batch	EC2/Fargate/ECS/EKS compute, containerised jobs	Not stated on page	None stated	Managed provisioning and queueing
Kubernetes [39]	Cluster	Extended resources via filtering/scoring plugins	Not stated	None stated	Filter, score, bind phases
Ray [40]	Distributed Python	Declared CPU/GPU resources; spread, node affinity, placement groups	Task graph by futures (not on page)	No runtime prediction stated	Hybrid default policy
Apache Spark [41]	Data processing	Executors; dynamic allocation	Stages by shuffle dependencies (not on page)	None stated	FIFO or FAIR; pools
Slurm [42]	HPC batch	GRES: GPUs, MPS, MIG devices as resources	Not on page	None stated	GPU requests by job script
StarPU [43]	Task runtime (Inria)	CPU/GPU tasks, heterogeneous scheduling, data transfers	Task graph	Performance models (not detailed on page)	Research runtime
Pegasus WMS [44]	Workflow manager	HTCondor pools, HPC, cloud; clustering of tasks	DAG workflows	None stated	Generator for the Montage instance used here

The comparison shows a division of labour. Operating-system and hardware features (Thread Director, Windows QoS, EAS) place *threads* on cores using hardware feedback or energy models but know nothing about application task graphs. Cluster and batch products (Kubernetes, AWS Batch, Slurm, Spark, Ray) schedule containers or jobs against declared resource requests; none of the cited pages describes execution-time prediction in the scheduler. Workflow and task runtimes (Pegasus, StarPU) understand DAGs; only StarPU is described as using performance models. NVIDIA Run:ai [45] is positioned for GPU orchestration, but only its landing page was read, so we make no feature claims. **No surveyed product combines application-level DAG knowledge, learned per-resource runtime estimates, and published decision-overhead measurements on a laptop-class hybrid CPU/GPU system.**

VI. Technology Trends

- **Hybrid and AI-accelerated client processors.** P/E cores are standard in recent client CPUs [25] and Copilot+ PCs require an NPU of at least 40 TOPS [46], so a laptop already exposes three engine types (CPU classes, GPU, NPU) whose placement the application may have to guide.
- **Edge growth.** Connected IoT devices grow by about 12–14% per year [23]; low-cost boards such as the Raspberry Pi 5 [14] (2.4 GHz Cortex-A76, Gigabit Ethernet, PCIe 2.0) are realistic remote workers for offloading experiments.
- **From heuristics to learning and back.** Learned schedulers progressed from DQN to GNN+PPO and meta-RL [1], [8], [11], [13], but a 2026 line of work replaces training with LLM agents plus runtime monitors [28], [47]; simultaneously, runtime-prediction work [26] shows that cheap models can approach perfect-knowledge scheduling in simulation.
- **Sustainability.** Data-centre electricity is projected to more than double by 2030 [21]; energy-aware scheduling (Linux EAS [37]) is already in production kernels.
- **Reproducible workflow research.** WfCommons [16] provides traces and generators for workflows such as Montage; evaluation on real executions rather than traces remains rare.
- **Overhead-aware evaluation.** Lightweight schedulers whose cost is measured [4] are increasingly motivated by fine-grained tasks; we expect decision overhead and statistical rigour to become standard reporting items.

TABLE VI

TECHNOLOGY TRENDS: INTRODUCTION AND ADOPTION STATUS (AS DOCUMENTED IN THE CITED SOURCES)

Trend	Introduced / state	Adoption status	Relevance to the project
Hybrid P/E cores with hardware scheduling hints	Thread Director listed for 12th–14th generation Core [25]	mainstream in client CPUs; OS-level use	the test machine has 6 P and 8 E cores; core type changes task time by 2.8–3.6 times
On-device accelerators (GPU, NPU)	NPU of 40 TOPS or more required for Copilot+ PCs [46]	shipping in consumer laptops	a laptop exposes three engine types; the project uses the CUDA GPU as one worker
Energy-aware scheduling	Linux EAS for asymmetric CPUs [37]	production kernels, with stated limits	context for a later energy metric; not measured here
Learned DAG scheduling (GNN, DRL)	LACHESIS to GNN+PPO and meta-RL [1], [13], [11]	research; evaluated in simulation (2 of 13 papers use hardware)	the project tests a much cheaper model on real execution
LLM-agent schedulers	2025–2026 preprints [28], [47]	early research, small DAGs	not evaluated; noted as a direction
Runtime performance models for task graphs	StarPU history models [20]; Lotaru [26]	established runtime; prediction results in simulation	same idea, tested with a ridge model and overhead accounting
Edge boards as remote workers	Raspberry Pi 5 [14]	widely available	emulated in the project; no real-link result

VII. CONCLUSION AND GAPS CARRIED FORWARD

The survey supports four conclusions. (1) Heterogeneous resources are abundant, growing and under-used, so scheduling matters. (2) The literature on DAG scheduling and offloading is dominated by simulated, learning-heavy evaluations; physical evidence exists in only two of 13 papers. (3) Patents disclose task graphs, queues, learned task ranking and duration prediction, but not an empirical method that accounts for the cost of prediction. (4) Products schedule threads or containers; none was found that schedules application DAGs with learned runtime estimates on a hybrid laptop.

The gaps carried into the next review are: **G1** real-hardware evidence for DAG scheduling on hybrid CPU+GPU(+remote) systems; **G2** accounting of the scheduler's own overhead; **G3** statistical reporting (paired intervals, multiplicity control); **G4** baselines that include informed simple rules and the HEFT/PEFT family; **G5** generalisation across workload instances; **G6** separation of prediction accuracy from scheduling value; **G7** real remote execution (Raspberry Pi) beyond simulation; **G8** a like-for-like comparison of static plans and dynamic dispatch with one cost model and end-to-end timing. The table also states the status after the project's final benchmark (Review 3).

TABLE VII

GAP ANALYSIS: EVIDENCE FROM THE SURVEY AND WHAT THE PROJECT DOES

Gap	Evidence in the survey	Project response and status
G1 real hardware	2 of 13 papers use physical devices; none runs a real scientific workflow	closed on one laptop: native Windows runtime, P/E pools, CUDA worker; real Montage and image workloads; six workloads in the final benchmark
G2 overhead	decision time reported in 4 of 13 papers	closed: per-dispatch and coordinator scheduling time logged; end-to-end makespan includes start-up and policy set-up
G3 statistics	no paper reports a confidence interval	closed: paired log-ratio bootstrap, Holm correction over ten pre-stated contrasts, frozen protocol
G4 baselines	HEFT in 5 papers, PEFT in 2, informed simple rule in 0	closed: B0–B5, EFT ladder, HEFT, PEFT, HEFT-E2E and a parameter-aware hand rule; the hand rule came within 1% of the dynamic policy
G5 generalisation	single workload instance per setting is common	partly closed: twenty unseen chain orders earlier, six workloads in the final benchmark; one workload family only
G6 accuracy vs value	accuracy and makespan rarely separated	closed: accurate remote predictions that gave no gain; GPU model error 23–31% reported next to scheduling results
G7 remote reality	simulated networks; one Jetson testbed	open: node/proxy path and loopback emulation only, labelled as such; no Raspberry Pi or real-network result
G8 static vs dynamic	HEFT used as a baseline in 5 of 13 papers, all in simulation	closed for this workload family: same ridge model on both sides; dynamic PRED faster than static HEFT by 5.2% (CPU) and 12.2% (strict GPU)

Threats to the survey. The 13 papers are the project's supplied base set, not a systematic review of the field; keyword counts approximate what a paper reports; patent summaries rely on Google Patents text and on our reading of the base patent's claims pages, not on a legal analysis; product statements are limited to what the cited pages state. The figures in Section II were checked against their sources, but two are secondary reports (IEA, TOP500). These limits are why we speak of "not found in the sources inspected" rather than "does not exist".

REFERENCES

[1] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, "Learning to optimize DAG scheduling in heterogeneous environment," in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[2] C. Sun, T. Yang, and Y. Lei, "DDDQN-TS: A task scheduling and load balancing method based on optimized deep reinforcement learning in heterogeneous computing environment," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9138–9172, 2022, doi: 10.1002/int.22983.

[3] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, "Dependent task offloading for edge computing based on deep reinforcement learning," *IEEE Trans. Comput.*, 2022, doi: 10.1109/TC.2021.3131040.

[4] A. Rodríguez, A. Navarro, K. Nikov, J. Nunez-Yanez, R. Gran, D. Suárez Gracia, and R. Asenjo, "Lightweight asynchronous scheduling in heterogeneous reconfigurable systems," *J. Syst. Archit.*, vol. 124, Art. no. 102398, 2022, doi: 10.1016/j.sysarc.2022.102398.

[5] Y. Song, C. Li, L. Tian, and H. Song, "A reinforcement learning based job scheduling algorithm for heterogeneous computing environment," *Comput. Electr. Eng.*, vol. 107, Art. no. 108653, 2023, doi: 10.1016/j.compeleceng.2023.108653.

[6] B. Gong and X. Jiang, "Dependent task-offloading strategy based on deep reinforcement learning in mobile edge computing," *Wireless Commun. Mobile Comput.*, 2023, doi: 10.1155/2023/4665067.

[7] J. Liu, Y. Mi, X. Zhang, and X. Li, "Task graph offloading via deep reinforcement learning in mobile edge computing," *Future Gener. Comput. Syst.*, vol. 158, pp. 545–555, 2024, doi: 10.1016/j.future.2024.04.034 (arXiv:2309.10569).

[8] Z. Cao, X. Deng, S. Yue, P. Jiang, J. Ren, and J. Gui, "Dependent task offloading in edge computing using GNN and deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21632–21646, 2024, doi: 10.1109/JIOT.2024.3374969.

[9] G. Nieto, I. de la Iglesia, U. Lopez-Novoa, and C. Perfecto, "Deep reinforcement learning techniques for dynamic task offloading in the 5G edge-cloud continuum," *J. Cloud Comput.*, vol. 13, Art. no. 94, 2024, doi: 10.1186/s13677-024-00658-0.

[10] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, "Dynamic task offloading in edge computing based on dependency-aware reinforcement learning," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.

[11] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, "Dynamic task offloading scheme for edge computing via meta-reinforcement learning," *Comput. Mater. Contin.*, vol. 82, no. 2, 2025, doi: 10.32604/cmc.2024.058810.

[12] C. Liu, H. Wang, M. Zhao, J. Liu, X. Zhao, and P. Yuan, "Dependency-aware online task offloading based on deep reinforcement learning for IoV," *J. Cloud Comput.*, vol. 13, Art. no. 136, 2024, doi: 10.1186/s13677-024-00701-0.

[13] Q. Tan, W. Chen, and D. Liu, "Task scheduling for heterogeneous multicore processors based on deep reinforcement learning," *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[14] Raspberry Pi Ltd, "Raspberry Pi 5 product page," https://www.raspberrypi.com/products/raspberry-pi-5/.

[15] Caltech/IPAC, "Montage: An astronomical image mosaic engine," software, https://github.com/Caltech-IPAC/Montage (commit ac0a1533, used in this work).

[16] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022, doi: 10.1016/j.future.2021.09.043.

[17] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proc. ACM SIGCOMM*, 2019 (arXiv:1810.01963).

[18] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[19] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[20] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: A unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).

[21] International Energy Agency, "Energy and AI," Paris, France, Apr. 2025 (figures as reported in the summary and press coverage; the primary report page was not opened in this work).

[22] TOP500, "June 2025 list: highlights," https://top500.org/lists/top500/2025/06/highs; accelerator counts and shares as analysed by The Next Platform, Jun. 10, 2025.

[23] IoT Analytics, "Number of connected IoT devices," https://iot-analytics.com/number-connected-iot-devices/ (18.5 billion at end of 2024; 21.1 billion in 2025).

[24] Y. Cheng, Z. Chai, and A. Anwar, "Characterizing co-located datacenter workloads: An Alibaba case study," arXiv:1808.02919, 2018.

[25] Intel, "What is Intel Thread Director?," Intel support article 000097053, https://www.intel.com/content/www/us/en/support/articles/000097053/processors.html.

[26] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.

[27] A. Ilyushkin and D. Epema, "The impact of task runtime estimate accuracy on scheduling workloads of workflows," in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGrid)*, 2018, pp. 331–341, doi: 10.1109/CCGRID.2018.00048.

[28] T. Lu and S. Reda, "Agentic CPU-GPU scheduling for heterogeneous AI workloads," arXiv:2607.22242, Jul. 2026.

[29] S. Nagpal, A. Karumannil, V. Jain, and A. K. Patil (Xilinx, Inc.), "Scheduling processing of machine learning tasks on heterogeneous compute circuits," U.S. Patent 11,561,826 B1, Jan. 24, 2023 (filed Nov. 12, 2020).

[30] A. Vega *et al.* (International Business Machines Corp.), "Heterogeneous system on a chip scheduler with learning agent," U.S. Patent 11,740,933 B2, Aug. 29, 2023.

[31] A. Amarnath *et al.* (International Business Machines Corp.), "Learning agent based application scheduling," U.S. Patent 11,966,776 B2, Apr. 23, 2024.

[32] R. Barik, T. Shpeisman, B. T. Lewis, and R. Kaleem (Intel Corp.), "Adaptive scheduling for task assignment among heterogeneous processor cores," U.S. Patent 10,186,007 B2, Jan. 22, 2019.

[33] R. Kim and M.-S. Kim (Samsung Electronics Co., Ltd.), "Method and apparatus for performing task scheduling in terminal," U.S. Patent 10,162,671 B2, Dec. 25, 2018.

[34] B. P. Englert, E. B. Harris, N. G. Crane, and B. J. Corey (Apple Inc.), "Scheduling heterogeneous execution on heterogeneous hardware," U.S. Patent 11,775,811 B2, Oct. 3, 2023.

[35] J. Wang and H. Huang (Google LLC), "Task scheduling for machine-learning workloads," U.S. Patent 11,544,113 B2, Jan. 3, 2023.

[36] Microsoft, "Quality of Service (Win32 apps)," Microsoft Learn, 2025, https://learn.microsoft.com/en-us/windows/win32/procthread/quality-of-service.

[37] The Linux Kernel documentation, "Energy Aware Scheduling," https://docs.kernel.org/scheduler/sched-energy.html.

[38] Amazon Web Services, "What is AWS Batch?," AWS Batch User Guide, https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html.

[39] The Kubernetes Authors, "Kubernetes Scheduler (kube-scheduler)," https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/.

[40] Anyscale/Ray Project, "Scheduling — Ray Core documentation," https://docs.ray.io/en/latest/ray-core/scheduling/index.html.

[41] Apache Software Foundation, "Job scheduling — Spark documentation," https://spark.apache.org/docs/latest/job-scheduling.html.

[42] SchedMD, "Generic Resource (GRES) scheduling," Slurm Workload Manager documentation, https://slurm.schedmd.com/gres.html.

[43] Inria, “StarPU: A unified runtime system for heterogeneous multicore architectures,” https://starpu.gitlabpages.inria.fr/.

[44] Pegasus Team, “Pegasus Workflow Management System documentation, v5.1.3,” https://pegasus.isi.edu/documentation/.

[45] NVIDIA, “NVIDIA Run:ai documentation,” https://run-ai-docs.nvidia.com/ (landing page only was read).

[46] Microsoft/Qualcomm, Copilot+ PC requirement of an NPU of at least 40 TOPS, as stated in vendor documents (e.g., Qualcomm, “What TOPS means: Snapdragon X Series”).

[47] X. Wei *et al.*, “Agent.xpu: Efficient scheduling of agentic LLM workloads on heterogeneous SoC,” arXiv:2506.24045, 2025.